MODUL PRAKTIKUM 4
PEMROGRAMAN BERORIENTASI OBJEK DASAR

4.1. Topik Bahasan

Modul ini dilaksanakan pada pertemuan ke-4 sesuai Rencana Pembelajaran Semester mata kuliah Pemrograman IV dan membebani CPMK-02, dengan cakupan topik sebagai berikut:

- Pembentukan kelas `Barang`, `Gudang`, dan `Pengiriman`.
- Penetapan atribut dan metode pada kelas.
- Konstruktor, konstruktor bernama, dan konstruktor pengalihan.
- Hak akses anggota kelas serta penggunaan getter dan setter.
- Pemetaan entitas proses logistik ke dalam rancangan kelas.

4.2. Capaian Pembelajaran

Mahasiswa mampu memodelkan entitas pada proses logistik ke dalam kelas dan objek (CPMK-02).

4.3. Indikator Ketercapaian Pembelajaran

Indikator penilaian pertemuan ke-4 menurut Rencana Pembelajaran Semester adalah:

- Ketepatan rancangan kelas beserta atribut dan metodenya
- Kesesuaian model kelas dengan entitas proses logistik

Indikator tersebut dinyatakan tercapai apabila bukti berikut terlihat pada hasil praktikum:

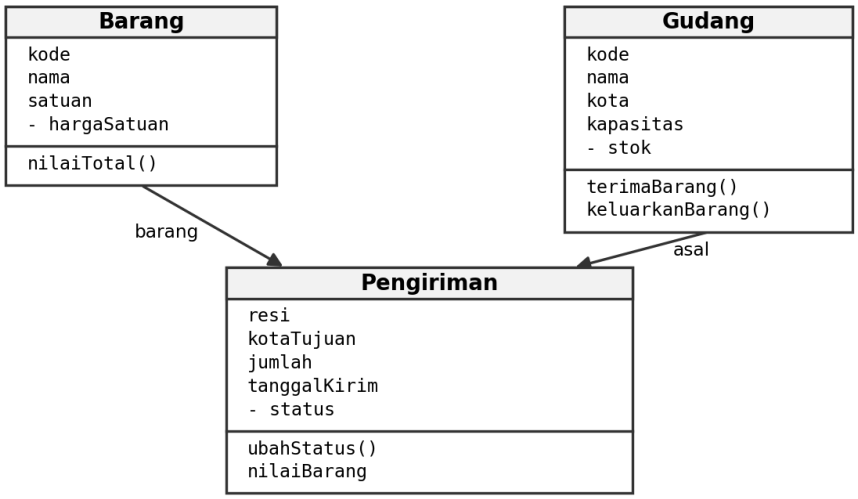
- Rancangan kelas sesuai dengan entitas yang terdapat pada kasus logistik yang dipilih.
- Objek terbentuk melalui konstruktor yang tepat, termasuk konstruktor bernama.
- Metode menghasilkan *output* yang benar untuk seluruh skenario uji.
- Atribut yang bersifat internal tidak dapat diubah langsung dari luar kelas.

4.4. Teori Pendukung

Kelas dan Objek

Pemrograman berorientasi objek memandang program sebagai kumpulan objek yang saling berinteraksi. Kelas adalah cetak biru yang mendefinisikan data (atribut) dan perilaku (metode) suatu konsep, sedangkan objek adalah wujud nyata kelas tersebut pada saat program berjalan. Dalam kasus logistik, `Barang` adalah kelas, sedangkan satu kardus berisi sepatu dengan kode B-1024 adalah objek dari kelas tersebut.

Perancangan kelas berangkat dari tiga pertanyaan: entitas apa saja yang ada pada proses bisnis, data apa yang melekat pada tiap entitas, dan tindakan apa yang dapat dilakukan entitas tersebut. Atribut menjawab pertanyaan kedua, sedangkan metode menjawab pertanyaan ketiga.



Gambar 4. Diagram kelas entitas Barang, Gudang, dan Pengiriman

Konstruktor

Konstruktor adalah metode khusus yang dijalankan saat objek dibentuk. Dart menyediakan penulisan ringkas melalui parameter inisialisasi `this.namaAtribut` sehingga tidak perlu menuliskan penugasan satu per satu. Konstruktor bernama memungkinkan sebuah kelas memiliki beberapa cara pembentukan objek, misalnya membentuk `Barang` dari data mentah hasil pemindaian kode atau membentuk `Pengiriman` yang baru didaftarkan dengan status awal tertentu.

Parameter bernama yang ditandai `required` memaksa pemanggil mengisi data wajib, sedangkan parameter bernama tanpa `required` boleh dikosongkan. Kombinasi ini membuat pembentukan objek jelas terbaca sekaligus aman terhadap data yang kurang lengkap.

Hak Akses, Getter, dan Setter

Dart tidak memiliki *keyword* `private` atau `public`. Hak akses ditentukan melalui penamaan: pengenal yang diawali garis bawah, misalnya `_stok`, hanya dapat diakses dari dalam berkas (*library*) yang sama. Mekanisme ini menjadi dasar penerapan enkapsulasi, yaitu menyembunyikan keadaan internal objek dan hanya membuka akses yang terkendali.

Akses terkendali disediakan melalui getter untuk membaca nilai turunan dan setter untuk mengubah nilai disertai pemeriksaan. Sebagai contoh, stok gudang tidak boleh

diubah langsung menjadi bilangan negatif; perubahan stok hanya boleh terjadi melalui metode `terimaBarang` dan `keluarkanBarang` yang memeriksa kewajaran nilainya.

Kode Program 4.1 Contoh getter dan setter

```
class Barang {
  final String kode;
  double _hargaSatuan;

  Barang(this.kode, this._hargaSatuan);

  double get hargaSatuan => _hargaSatuan;

  set hargaSatuan(double nilai) {
    if (nilai < 0) {
      throw ArgumentError('Harga satuan tidak boleh negatif');
    }
    _hargaSatuan = nilai;
  }
}
```

Metode toString

Setiap kelas pada Dart mewarisi metode `toString()` dari kelas `Object`. Menuliskan ulang metode ini dengan penanda `@override` membuat objek dapat dicetak dalam bentuk yang mudah dibaca, sehingga memudahkan *debugging*. Kebiasaan ini akan terus dipakai hingga modul-modul berikutnya.

Tabel 5. Pemetaan entitas proses logistik ke dalam rancangan kelas

Entitas nyata	Kelas	Atribut utama	Perilaku utama
Barang di gudang	`Barang`	kode, nama, satuan, harga satuan	menghitung nilai total sejumlah unit
Gudang penyimpanan	`Gudang`	kode, nama, kota, kapasitas, stok	menerima dan mengeluarkan barang
Pengiriman barang	`Pengiriman`	resi, kota tujuan, jumlah, tanggal, status	mengubah status sesuai urutan yang sah
Pelanggan	`Pelanggan`	kode, nama, telepon, alamat	menyajikan alamat ringkas

4.5. Alat Pendukung

Perangkat keras, perangkat lunak, dan bahan yang diperlukan pada modul ini adalah:

- Flutter SDK dan Dart SDK.
- Visual Studio Code beserta ekstensi Flutter dan Dart.
- Terminal PowerShell.
- Perangkat lunak pembuat diagram (draw.io atau Figma) untuk menggambar diagram kelas.
- Repositori GitHub pribadi praktikan (satu repositori untuk seluruh modul praktikum).

4.6. Langkah-langkah Praktikum

A. Penyiapan dan Perancangan

1. Buka proyek `silog_app` melalui menu File > Open Folder pada Visual Studio Code, lalu buka terminal terintegrasi melalui menu Terminal > New Terminal. Buat cabang kerja modul ini pada terminal dengan perintah `git checkout -b praktikum/modul-04`, kemudian pastikan cabang aktif melalui `git branch`.
2. Buat direktori `lib/model` melalui panel Explorer: klik kanan direktori `lib`, pilih New Folder, lalu ketik `model`.
3. Gambarkan diagram kelas sederhana untuk `Barang`, `Gudang`, dan `Pengiriman` beserta atribut dan metodenya melalui draw.io pada alamat `https://app.diagrams.net`, gunakan menu File > New > Blank Diagram lalu bentuk UML pada panel Shapes. Simpan hasilnya sebagai gambar untuk dilampirkan pada jurnal.

B. Kelas Barang

1. Buat berkas `lib/model/barang.dart` melalui panel Explorer: klik kanan direktori `lib/model`, pilih New File, lalu ketik nama berkas `barang.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 4.2 Berkas lib/model/barang.dart

```dart
class Barang {
  final String kode;
  final String nama;
  final String satuan;
  double _hargaSatuan;
  double beratKg;

  Barang({
    required this.kode,
    required this.nama,
    required double hargaSatuan,
    this.satuan = 'pcs',
    this.beratKg = 0,
  }) : _hargaSatuan = hargaSatuan;

  /// Konstruktor bernama untuk data hasil pemindaian kode barang.
  Barang.dariPemindaian(String baris)
      : kode = baris.split(';')[0],
        nama = baris.split(';')[1],
        satuan = 'pcs',
        _hargaSatuan = double.parse(baris.split(';')[2]),
        beratKg = 0;

  double get hargaSatuan => _hargaSatuan;

  set hargaSatuan(double nilai) {
    if (nilai < 0) {
      throw ArgumentError('Harga satuan tidak boleh negatif');
    }
    _hargaSatuan = nilai;
  }

  double nilaiTotal(int jumlah) => _hargaSatuan * jumlah;
```

```
  @override
  String toString() => '[$kode] $nama ($satuan) -
Rp${_hargaSatuan.toStringAsFixed(0)}';
}
```

C. Kelas Gudang

1. Buat berkas `lib/model/gudang.dart` melalui panel Explorer: klik kanan direktori `lib/model`, pilih New File, lalu ketik nama berkas `gudang.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 4.3 Berkas lib/model/gudang.dart

```
import 'barang.dart';

class Gudang {
  final String kode;
  final String nama;
  final String kota;
  final int kapasitas;

  final Map<String, int> _stok = {};

  Gudang({
    required this.kode,
    required this.nama,
    required this.kota,
    this.kapasitas = 1000,
  });

  int get totalStok => _stok.values.fold(0, (a, b) => a + b);

  bool get penuh => totalStok >= kapasitas;

  Map<String, int> get stok => Map.unmodifiable(_stok);

  void terimaBarang(Barang barang, int jumlah) {
    if (jumlah <= 0) {
      throw ArgumentError('Jumlah penerimaan harus lebih besar dari nol');
    }
    if (totalStok + jumlah > kapasitas) {
      throw StateError('Kapasitas gudang $nama tidak mencukupi');
    }
    _stok[barang.kode] = (_stok[barang.kode] ?? 0) + jumlah;
  }

  void keluarkanBarang(Barang barang, int jumlah) {
    final tersedia = _stok[barang.kode] ?? 0;
    if (jumlah > tersedia) {
      throw StateError(
          'Stok ${barang.kode} hanya $tersedia, tidak dapat mengeluarkan $jumlah');
    }
    _stok[barang.kode] = tersedia - jumlah;
  }

  @override
  String toString() => '$kode - $nama ($kota), stok $totalStok/$kapasitas';
}
```

D. Kelas Pengiriman

1. Buat berkas `lib/model/pengiriman.dart` melalui panel Explorer: klik kanan direktori `lib/model`, pilih New File, lalu ketik nama berkas `pengiriman.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 4.4 Berkas lib/model/pengiriman.dart

```dart
import 'barang.dart';
import 'gudang.dart';

class Pengiriman {
  final String resi;
  final Gudang asal;
  final String kotaTujuan;
  final Barang barang;
  final int jumlah;
  final DateTime tanggalKirim;

  String _status;

  Pengiriman({
    required this.resi,
    required this.asal,
    required this.kotaTujuan,
    required this.barang,
    required this.jumlah,
    required this.tanggalKirim,
    String status = 'Draft',
  }) : _status = status;

  /// Konstruktor bernama untuk pengiriman yang baru didaftarkan hari ini.
  Pengiriman.baru({
    required this.resi,
    required this.asal,
    required this.kotaTujuan,
    required this.barang,
    required this.jumlah,
  })  : tanggalKirim = DateTime.now(),
        _status = 'Menunggu Pickup';

  String get status => _status;

  double get nilaiBarang => barang.nilaiTotal(jumlah);

  void ubahStatus(String statusBaru) {
    const urutan = [
      'Draft',
      'Menunggu Pickup',
      'Dalam Perjalanan',
      'Tiba di Kota Tujuan',
      'Diterima',
    ];
    if (!urutan.contains(statusBaru)) {
      throw ArgumentError('Status $statusBaru tidak dikenal');
    }
    if (urutan.indexOf(statusBaru) < urutan.indexOf(_status)) {
      throw StateError('Status tidak dapat mundur dari $_status ke $statusBaru');
    }
    _status = statusBaru;
  }

  @override
  String toString() =>
```

```
      '$resi | ${barang.nama} x$jumlah | ${asal.kota} -> $kotaTujuan | $_status';
}
```

E. Pengujian Model

1. Buat berkas `latihan/modul04_uji.dart` melalui panel Explorer: klik kanan direktori `latihan`, pilih New File, lalu ketik nama berkas `modul04_uji.dart`. Ketik kode berikut, kemudian simpan melalui `Ctrl+S`.

Kode Program 4.5 Pengujian rancangan kelas

```dart
import '../lib/model/barang.dart';
import '../lib/model/gudang.dart';
import '../lib/model/pengiriman.dart';

void main() {
  final sepatu = Barang(
    kode: 'B-1024',
    nama: 'Sepatu Lari Pria',
    hargaSatuan: 450000,
    beratKg: 0.9,
  );
  final kaos = Barang.dariPemindaian('B-2048;Kaos Katun;85000');

  final gudangBandung = Gudang(
    kode: 'GD-BDG',
    nama: 'Gudang Utama Bandung',
    kota: 'Bandung',
    kapasitas: 500,
  );

  gudangBandung.terimaBarang(sepatu, 120);
  gudangBandung.terimaBarang(kaos, 300);
  print(gudangBandung);
  print('Rincian stok: ${gudangBandung.stok}');

  final kirim = Pengiriman.baru(
    resi: 'SLG-2026-0007',
    asal: gudangBandung,
    kotaTujuan: 'Surabaya',
    barang: sepatu,
    jumlah: 10,
  );

  gudangBandung.keluarkanBarang(sepatu, kirim.jumlah);
  print(kirim);
  print('Nilai barang: Rp${kirim.nilaiBarang.toStringAsFixed(0)}');

  kirim.ubahStatus('Dalam Perjalanan');
  print('Status terbaru: ${kirim.status}');

  try {
    kirim.ubahStatus('Draft');
  } on StateError catch (e) {
    print('Ditolak: ${e.message}');
  }

  try {
    gudangBandung.keluarkanBarang(sepatu, 9999);
  } on StateError catch (e) {
    print('Ditolak: ${e.message}');
  }
```

```
}
```

2. Jalankan program melalui `dart run latihan/modul04_uji.dart` dan catat seluruh output-nya.
3. Cobalah mengubah nilai `_stok` secara langsung dari `main()`, amati pesan *error* dari *compiler*, lalu jelaskan hubungannya dengan enkapsulasi.

F. Tugas Praktikum

1. Tambahkan kelas `Pelanggan` yang memiliki atribut kode, nama, nomor telepon, dan alamat, serta metode `alamatRingkas()` yang mengembalikan alamat maksimal 40 karakter.
2. Tambahkan atribut `pelanggan` pada kelas `Pengiriman` dan sesuaikan metode `toString()`.
3. Lampirkan diagram kelas akhir pada jurnal dan jelaskan alasan pemilihan atribut yang dijadikan `final` serta atribut yang dijadikan privat.

G. Luaran yang Dinilai

Bentuk penilaian modul ini adalah tugas praktikum dengan kriteria kesesuaian rancangan kelas dan kebenaran implementasi kode. jurnal praktikum individu diunggah ke lms ulbi paling lambat 5x24 jam setelah sesi praktikum. Agar kriteria tersebut dapat dinilai, praktikan menyerahkan luaran berikut:

- Diagram kelas `Barang`, `Gudang`, `Pengiriman`, dan `Pelanggan` beserta atribut, metode, dan relasinya.
- Berkas model pada direktori `lib/model`.
- Tangkapan layar output `modul04_uji.dart` yang memperlihatkan pembentukan objek, perubahan stok, dan penolakan operasi yang tidak sah.
- Penjelasan pemetaan tiap kelas ke entitas nyata pada proses logistik, serta alasan pemilihan atribut `final` dan atribut privat.

4.7. Panduan Pengumpulan Jurnal Hasil Kegiatan Praktikum

Tugas praktikum modul ini bersifat individu. Setiap praktikan menyusun dan mengunggah sendiri jurnalnya ke LMS ULBI paling lambat 5 x 24 jam setelah sesi praktikum berakhir, berupa berkas PDF bernama `NIM_Nama_Modul-XX.pdf`, disertai tautan repositori GitHub pribadi pada cabang `praktikum/modul-XX`.

Sistematika jurnal:

1. Identitas: nama, NIM, kelas, nomor dan judul modul, serta tanggal praktikum.
2. Tujuan praktikum.
3. Alat dan bahan yang digunakan beserta versinya.
4. Langkah kerja, disertai tangkapan layar tiap tahap penting.
5. Kode program beserta penjelasan singkatnya.

6. Hasil eksekusi dan analisisnya, termasuk error yang ditemui beserta penyelesaiannya.
7. Jawaban seluruh butir Tugas Praktikum.
8. Kesimpulan dan kendala.
9. Tautan repositori beserta tangkapan layar riwayat *commit*.

Tangkapan layar wajib memperlihatkan jendela *browser* atau emulator secara utuh beserta kode programnya, dan memuat nama serta NIM praktikan. Praktikan yang tidak mengumpulkan jurnal memperoleh nilai nol; keterlambatan dan ketidaksesuaian hasil lainnya dikenai pengurangan nilai sesuai kebijakan dosen pengampu.

Seluruh butir pada sub bab Luaran yang Dinilai wajib dilampirkan pada jurnal.

4.8. Hasil Capaian Pembelajaran Praktikum

Bentuk dan kriteria penilaian mengikuti Rencana Pembelajaran Semester pertemuan ke-4: Tugas praktikum: kesesuaian rancangan kelas dan kebenaran implementasi kode. Jurnal praktikum individu diunggah ke LMS ULBI paling lambat 5x24 jam setelah sesi praktikum.

Tugas praktikum modul ini bersifat individu dan dinilai dari jurnal praktikum masing-masing praktikan.

Bobotnya 2% dari nilai akhir mata kuliah pada komponen evaluasi Praktikum (Aktivitas Partisipatif) sebesar 15% dari nilai akhir, diukur dari kelengkapan dan mutu luaran pada sub bab Luaran yang Dinilai terhadap indikator penilaian modul ini.